

A Four-Layer Modeling Framework for Policy-Driven Design in Hyperledger Fabric

Noor Mohammed Sabr Al-Gburi*, András Földvári*, Kristóf Marussy*, Oszkár Semeráth*, and Imre Kocsis*

* Budapest University of Technology and Economics, Department of Artificial Intelligence and Systems Engineering, Budapest, Hungary

ABSTRACT Distributed ledger technologies (DLTs), such as Hyperledger Fabric, are increasingly adopted in scenarios requiring multi-organizational collaboration without a central authority. Designing these architectures is error-prone because architectural violations only become visible when the complete specification is validated, leading to costly late stage redesigns. In this paper, we present a structured four-layer modeling approach that enables validation of partial designs, detecting architectural violations incrementally as engineers build models layer by layer. Our 4-layer metamodel explicitly separates consortium structure (Layer 0), network architecture (Layer 1), node topology (Layer 2), and infrastructure deployment (Layer 3), with cross-layer validation predicates. Our work demonstrates that Refinery's partial model reasoning capabilities can support such incremental validation workflows in practice. We show that endorsement policy violations, participation mismatches, and trust distribution issues can be detected at Layers 1-2 before infrastructure planning begins at Layer 3. We demonstrate detection of five violation classes (unrealized intents, single-organization channels, policy infeasibility, participation mismatches, cross-channel endorsements) in partial TPC-C supply chain models. Validation time remains interactive (<1 second) for models ranging from 5 to 20 organizations across all layers, enabling real-time feedback during incremental model construction. This validation-centric workflow reduces costly design errors in blockchain network architectures.

KEYWORDS Partial Model Validation, Distributed Ledger Technologies, Incremental Validation, HyperLedger Fabric, Cross-Layer Constraints.

1. Introduction

1.1. Context and Problem

Distributed ledger technologies (DLTs) (DiMatteo et al. 2019) such as Hyperledger Fabric are increasingly used in domains where multiple organizations must collaborate without relying on a central authority. Their configurable architecture enables designers to tailor channels, peers, endorsement strategies, and deployment choices to application-specific requirements. Endorsement policies, in particular, determine which organizations must approve transactions, and therefore directly affect resilience, privacy, and performance.

Architectural decisions in Fabric span multiple abstraction layers, including organizational structures, network topology, chaincode deployment, and endorsement configurations. These layers are interdependent; for example, endorsement policies specified at the network architecture layer may become infeasible when organizational constraints limit channel participation, or when the infrastructure layer reveals that an organization lacks resources to deploy the required endorsing peers. Current practices lack support for validating these interdependencies early, which often leads to late discovery of inconsistencies and costly redesigns. Manual reasoning across layers is error-prone and inefficient, especially when exploring multiple alternatives.

1.2. Objectives

Our paper aims to facilitate early, iterative validation of DLT architectures by combining layered modeling with automated reasoning. We propose a four-layer metamodel with cross-layer validation predicates, implemented in Refinery, that enables incremental checking of partial designs as engineers build models

JOT reference format:

Noor Mohammed Sabr Al-Gburi, András Földvári, Kristóf Marussy, Oszkár Semeráth, and Imre Kocsis. *A Four-Layer Modeling Framework for Policy-Driven Design in Hyperledger Fabric*. Journal of Object Technology. Vol. vv, No. nn, yyyy. Licensed under Attribution - NonCommercial - No Derivatives 4.0 International (CC BY-NC-ND 4.0)
<http://dx.doi.org/10.5381/jot.yyyy.vv.nn.aa>

layer by layer. Our goal is to enable designers to express partial decisions, assess their feasibility across layers, and receive constructive feedback.

Endorsement policies are modeled as formal constraints within an auditable framework, allowing systematic analysis of their implications alongside structural and deployment requirements. We evaluate our approach along three dimensions:

RQ1: Early Detection Capability Can the layered approach detect architectural violations in partial models before design completion?

RQ2: Interactive Feedback. Does validation time remain interactive (<1 second) during incremental model construction?

RQ3: Scalability. How does validation time scale with organizational count and model size?

1.3. Contributions

Building on prior work (Al-Gburi et al. 2024), which focused on single-step model generation, this paper introduces a multi-step refinement workflow, where each layer has its own semantics, constraints, and verification responsibilities. The key contributions are:

(1) A structured four-layer modeling approach that separates organizational, network, chaincode, and deployment concerns, enabling early error detection through continuous verification at each refinement step.

(2) Integration of endorsement policies as first-class architectural constraints from Layer 1, enabling continuous verification across all refinement steps. Unlike prior work, which treats endorsement policies as post-hoc deployment configurations (Androulaki et al. 2019; Sukhwani et al. 2018), our approach surfaces policy-related violations before node assignment or infrastructure planning begins.

(3) Layer-specific analysis capabilities including an ASP-based fault tolerance analysis that evaluate functional and extra-functional properties (EFP), including endorsement policy implications, at appropriate abstraction levels.

(4) A demonstration on HLF, illustrating how the approach supports early detection of inconsistencies and impact analysis of endorsement configurations. A demonstration that Refinery’s constraint language and partial models semantics are well-suited for expressing cross-layer architectural predicates in a domain (distributed ledger design) not previously addressed by graph-based model validation tools. The complete 4-layer metamodel, TPC-C instance models, ASP analysis scripts, benchmark generation scripts, and the Refinery configuration used for this evaluation are available at: <https://github.com/ftsrg/policy-driven-hlf>.

2. Background

2.1. Hyperledger Fabric Architecture

In this paper, we work with the Hyperledger Fabric (HLF) design language depicted in Figure 1. A Fabric network is jointly operated by a set of organizations. The network maintains a set of blockchain-backed distributed ledgers, called channels in Fabric. Each channel is associated with a set of participating organizations; each organization participating in a channel provides network nodes for the operation of the channel on

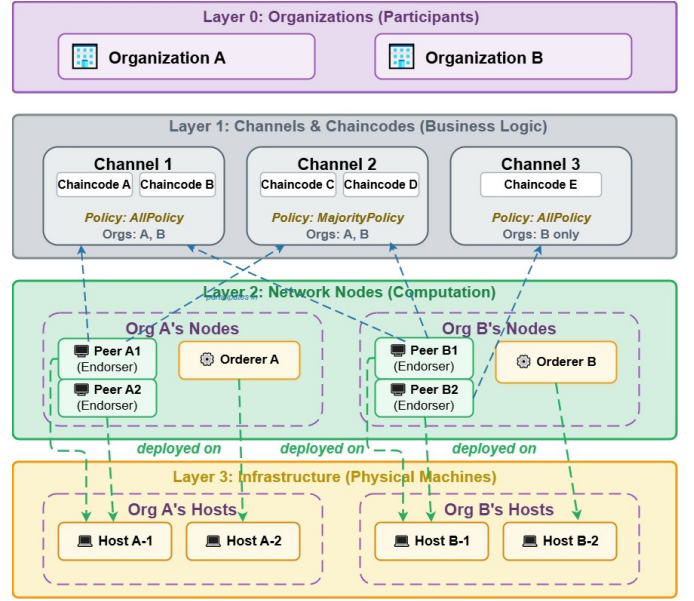


Figure 1 The Multi-Layer Architecture of HLF Network

its organizational host machines. Nodes either participate in ordering the transactions of the channel, or computing their side effects (“endorsing”).

Out of the box, Fabric provides only a simple versioned key-value ledger (channel) abstraction, without any native (built-in) asset or transaction type. Chaincode in Fabric (smart contracts) define the transaction types and implement them in general-purpose programming languages, relying on a key get/set style API.

Chaincode is instantiated on a channel by deployment to the endorsing nodes of a channel. Subsequently, transaction processing follows an Execute-Order-Validate pattern. First, an organizational client requests execution of a chaincode method across a number of endorsing nodes using their current (channel) ledger view, without actually modifying the ledger. If the client can collect enough matching replies on the “simulated” write effects, it submits those “endorsements” to the ordering service.

The ordering service orders endorsed transaction proposals, forms the next block of the blockchain and distributes it to the endorsing nodes. In Raft-based ordering (Ongaro & Ousterhout 2014) peer-to-peer consensus protocol.

2.2. Refinery

Refinery (Marussy et al. 2024) is a logic-based model generation and analysis tool that operates on partial models, which represent abstractions where some concepts or details are intentionally left unspecified or unknown, allowing for flexible modeling at different levels of abstraction (Semerath et al. 2019). Refinery’s constraint language supports the declaration of classes, references, and error predicates (*graph pattern constraints*) that are checked over partial models. When a predicate match is found, Refinery reports it as a violation, enabling validation of incomplete designs. Refinery supports bidirectional (opposite) references that propagate constraints across related model elements, and a (count) aggregation operator for quantitative pred-

icates such as majority checks. We selected Refinery for our approach because its partial-model semantics directly match our requirement of validating architectural designs at intermediate refinement stages, before all decisions have been committed.

2.3. Answer Set Programming

Answer Set Programming (ASP) (Lifschitz 2019; Calimeri et al. 2019) is a declarative formalization approach to model and solve search and optimization problems. ASP is based on the stable model semantics of logic programming (Gelfond & Lifschitz 1988). The concept of a stable model is used to define declarative semantics for logic programs with negation as failure.

An ASP programs consist of *atoms* of the form $\sigma(x_1, \dots, x_n)$, where $\sigma \in \Sigma$ is a logical symbol and $x_1, \dots, x_n$ are variables or objects constants, as well as *literals*, and *rules* of the form

$$r :- a_1, \dots, a_n, \text{not } b_1, \dots, \text{not } b_m.$$

The rule above is derived, meaning that the head (r) is true, when all the atoms and literals in the body (a_i, b_i) are true in the following sense: a non-negated literal a is true if the atom a has a derivation (from another rule or as a fact). A negated literal, $\text{not } b_i$, is true according to *negation as failure* semantics if the atom b does not have derivation. *Facts* are rules an empty body. The derivation of facts is always true.

The results of ASP are *answer sets* (stable models) that satisfy the program, i.e., constitute a consistent assignment of truth values to the atoms in the program according to the prescribed rules.

ASP tools often enhance the core semantics with additional *statements*. An *aggregate statement* (e.g., `#sum`, `#count`) applies to an ASP atom set and returns a number. Special *optimization statements* are used to specify optimization criteria to be maximized (`#maximize`) or minimized (`#minimize`) when finding solutions to a problem. These criteria are expressed as integer values associated with the answer sets, and ASP solvers aim to find solutions that optimize these values according to the specified criteria.

The `#show` statement specifies the *projection* of logical symbols to display in answer sets. It allows users to request specific information about the solution(s) produced by the solver, making it easier to analyze and interpret the results.

3. Motivating Example (Running Example)

3.1. Example Scenario

We demonstrate our approach using a blockchain-based supply chain consortium modeled after the TPC-C benchmark (Transaction Processing Performance Council 2010). The consortium consists of five organizations:

- **Manufacturer:** Produces goods.
- **Distributor:** Wholesale distribution.
- **Retailer:** Retail sales.
- **Courier:** Delivery services.
- **Regulator:** Regulatory oversight.

These organizations collaborate across four channels, each realizing a specific business intent:

1. **ChCommon:** Represents common reference data sharing and hosts CC_Warehouse and CC_District chaincodes.
2. **ChInventory:** Represents inventory tracking and hosts CC_Item, CC_Order, CC_Stock, and CC_OrderLine chaincodes.
3. **ChNewOrders:** Represents order processing and hosts CC_NewOrder chaincode.
4. **ChCustomer:** A highly restricted channel between the Retailer and Courier which hosts CC_Customer and CC_Payment chaincodes.

The TPC-C scenario is decomposed into four functional channels with differentiated participation and endorsement requirements (Table 1). This differentiation, varying from simple bilateral agreements in ChCustomer with 2 organizations, AllPolicy to full consortium consensus in ChCommon with 5 organizations, AllPolicy, enables our validation engine across varying topological complexity and policy constraints.

Table 1 TPC-C Channel Participation and Endorsement Constraints

Channel	Participants	Policy	Implication
ChCommon	All 5 orgs	All	All must endorse
ChInventory	4 Orgs (no Courier)	Majority	≥ 3 of 4 must endorse
ChNewOrders	3 Orgs (M, D, R)	Majority	≥ 2 of 3 must endorse
ChCustomer	2 Orgs (Ret, Courier)	All	Both must endorse

For example, ChInventory imposes a high trust threshold (*MajorityPolicy*) that becomes a "negative partial pattern" if the engineer fails to assign endorsing nodes to at least three out of the four participating organizations.

We will reference this running example throughout Section 4, demonstrating how violations are detected at each layer of the metamodel.

4. Layered Architecture Model

We decompose HLF architecture design into four layers based on the natural decision points that architects encounter in practice. Each layer corresponds to a distinct class of design decisions and introduces concepts that depend on, but do not modify, the abstractions established at higher layers. Layer 0 captures the organizational landscape: which entities participate and what business goals motivate the network. Layer 1 introduces technical structure: channels, chaincodes, and endorsement policies that realize the business intents. Layer 2 assigns concrete software components (*ordering and endorsing nodes*) to the structures defined at Layer 1. Layer 3 maps these components to physical or virtual infrastructure.

This decomposition reflects the temporal ordering of design decisions in practice: organizational agreements precede technical design, which precedes node assignment, which precedes infrastructure provisioning. Crucially, errors introduced at earlier layers compound through subsequent layers: an infeasible endorsement policy at Layer 1 propagates to Layer 2 as an unsatisfiable endorsement assignment, which in turn blocks infrastructure planning at Layer 3.

Figure 2 shows both the metamodel (left) and a concrete TPC-C instance (right) that conforms to it. Each horizontal band corresponds to one layer; the vertical dashed line separates abstract metamodel classes from their TPC-C instantiation. We now describe each layer's metamodel elements and the cross-layer predicates checked at each transition.

The complete TPC-C model instance is also visualized as Refinery graph models in Figures 5, 6, 7. Figure 5 depicts the Layer 0 consortium structure collaboration intents, organizations, hosts, and their network connectivity. Figure 6 shows Layers 1-2, including channels, chaincodes, endorsement policies, orderers, endorsers, and their deployment mappings. Figure 7 presents the complete model, connecting the elements of Figures 5 and 6 into a single graph. These figures show the model as Refinery internally represents it: nodes correspond to model elements (organizations, channels, chaincodes, etc.) and edges correspond to references between them.

4.1. Layer 0: Consortium Structure

Layer 0 represents the organizational landscape and the high-level business goals of the stakeholders. It captures the *why* of the system without technical implementation details.

Listing 1 illustrates the key entities and a sample consistency check for this layer.

```
class Consortium {
    contains Organization[1..*] members
    contains CollaborationIntent[1..*] intents
}
class Organization {
    contains Host[1..*] hosts
}
class CollaborationIntent {
    Organization[2..*] participants
    Channel[0..1] realizedBy opposite realizes
}
error participantNotInConsortium
(CollaborationIntent ci, Organization o) <=>
participants(ci, o),
intentInConsortium(ci, c),
!orgInConsortium(o, c).
```

Listing 1 Snippet from Layer 0: Organizations and Intents

The Consortium class represents the root entity that contains all organizations and intents. The Organization class represents a real-world entity participating in the consortium. The CollaborationIntent class establishes the "business rationale" for a blockchain network, requiring at least two participants to justify a decentralized ledger.

The realizedBy reference establishes the so-called "Golden Thread" to Layer 1, ensuring every business requirement is eventually mapped to a technical Channel.

Crucially, the participantNotInConsortium error predicate demonstrates our *early capture* methodology. It identifies a stakeholder-level configuration error attempting to include an external organization in a private intent before any network topology or chaincode logic is drafted.

In Figure 2, the top-left band shows the three metamodel classes (Consortium, Organization, CollaborationIntent) and their associations. The top-right band shows the TPC-C instantiation: SupplyChainConsortium with five organizations and four collaboration intents, each specifying its participant subset.

Running Example. In our TPC-C scenario, the consortium includes five organizations (Manufacturer, Distributor, Retailer, Courier, Regulator) with four collaboration intents (ReferenceDataSharing, InventoryTracking, OrderProcessing, CustomerFulfillment). Refinery validates that all intent participants are consortium members before proceeding to Layer 1.

Violation 1: Unrealized Collaboration Intent Detected at: Layers 0-1, before any nodes assigned. This violation occurs when an engineer specifies CollaborationIntent but lacks a corresponding technical Channel for implementation. Refinery detects unrealizedIntent(DataSharing), catching forgotten business requirements before any technical design work. Engineers avoid it before investing effort in designing node topology by either creating the missing channel or removing the unused intent.

4.2. Layer 1: Network Architecture

Layer 1 maps CollaborationIntents to technical Channels, deploys ChaincodeInstances, and assigns endorsement Policies. This is where business intent meets Fabric-specific technical concepts.

```
class Channel {
    contains ChaincodeInstance[1..*] chaincodes
}
CollaborationIntent[0..1] realizes opposite realizedBy
class ChaincodeInstance {refers Policy[1] policy
}
EndorsingNode[2..*] endorsedBy opposite endorses
abstract class Policy {}
class AnyPolicy extends Policy {}
class MajorityPolicy extends Policy {}
class AllPolicy extends Policy {}
error conflictingPolicies(ChaincodeInstance ci)
<=> policy(ci, p1), policy(ci, p2), p1 != p2.
error unrealizedIntent(CollaborationIntent ci)
<=> CollaborationIntent(ci), !realizedBy(ci, _).
```

Listing 2 Layer 1 snippet: Policy validation logic

Listing 2 shows early validation before any nodes are assigned. Refinery validates that each chaincode exactly one policy and all CollaborationIntents are realized as Channels.

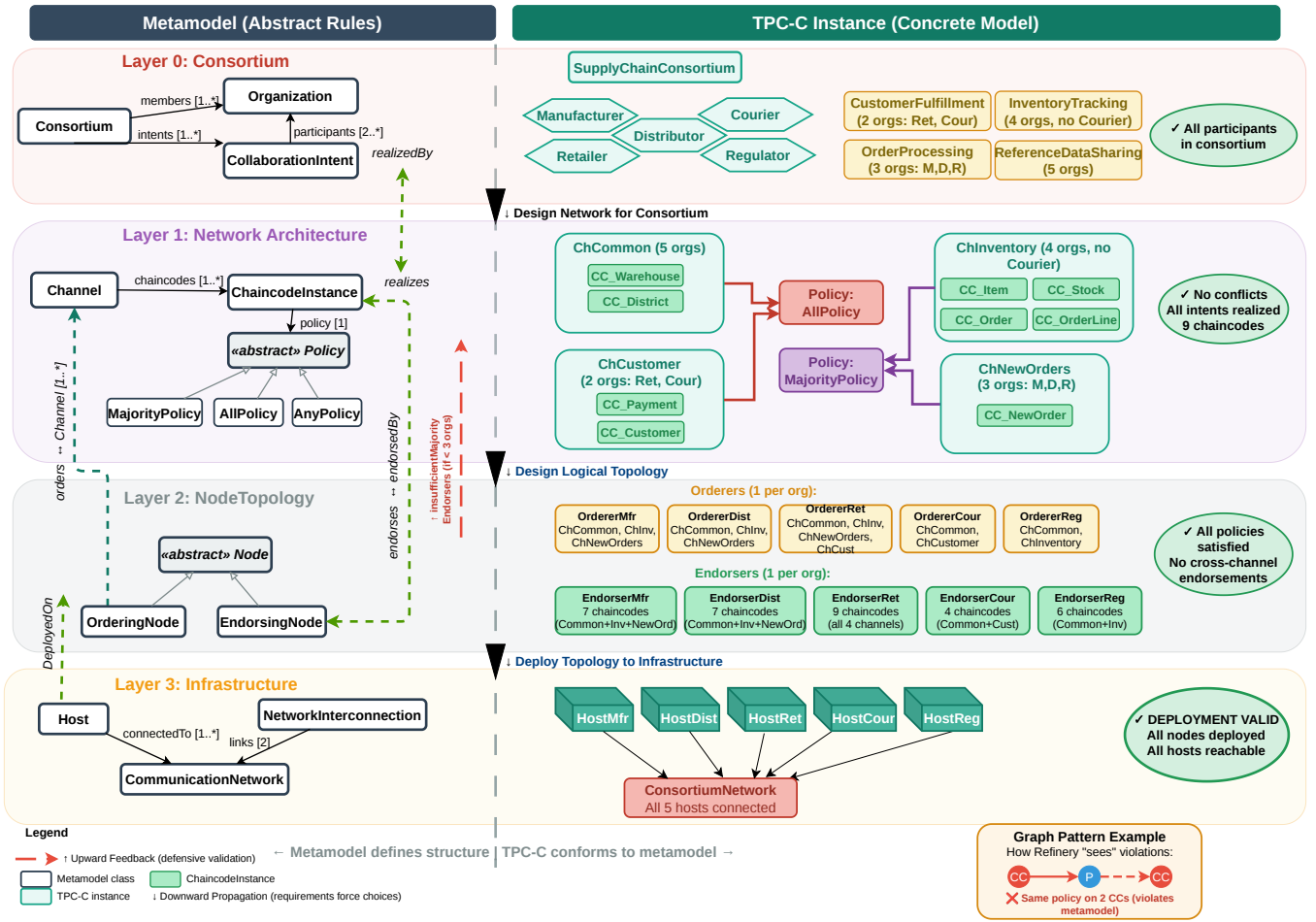


Figure 2 Four-layer refinement pipeline with metamodel-instance mapping

The second band of Figure 2 shows the `Channel`, `ChaincodeInstance`, and `Policy` classes on the left. On the right, the four TPC-C channels are shown with their chaincodes and assigned policies (*AllPolicy* for *ChCommon* and *ChCustomer*, *MajorityPolicy* for *ChInventory* and *ChNewOrders*).

Violation 2: Endorsement Policy Infeasibility Detected at: Layers 1-2 when policies are specified and before infrastructure.

This violation occurs when an engineer specifies `PolicyMajority` on a channel with only 2 participating organizations. The majority requires both to endorse, making the policy vulnerable to single-point failure. If any organization becomes unavailable, the policy cannot be satisfied. The engineer should consider `PolicyAll` (explicit) or add a third organization. This warning catches subtle policy design issues before node assignment.

4.3. Layer 2: Node Topology

Layer 2 specifies the software components (Nodes) and their specific roles: `OrderingNodes` provide consensus; `EndorsingNodes` execute chaincode. This layer ensures organizations provide the necessary technical presence to fulfill their logical roles defined in Layer 1.

```
abstract class Node {Host[1] deployedOn }
class OrderingNode extends Node {Channel[1..*]
orders}
class EndorsingNode extends Node
{ChaincodeInstance[1..*] endorses opposite
endorsedBy}

// Cross-layer predicates
pred nodeParticipatesInChannel(Node n, Channel
c) <=> OrderingNode(n), orders(n, c);
EndorsingNode(n), endorses(n, ci),
chaincodes(c, ci).
pred orgParticipatesInChannel(Organization o,
Channel c) <=> hosts(o, h), deployedOn(n, h),
nodeParticipatesInChannel(n, c).
pred channelHasMultipleOrgs(Channel c) <=>
orgParticipatesInChannel(o1, c),
orgParticipatesInChannel(o2, c), o1 != o2.
error singleOrgChannel(Channel c) <=>
Channel(c), !channelHasMultipleOrgs(c).
error channelWithoutOrderer(Channel c) <=>
Channel(c), !orders(_, c).

// Majority policy check
pred orgEndorsesChaincode(Organization o,
```

```

126 ChaincodeInstance ci) <-->
277 endorses(n, ci), deployedOn(n, h), hosts(o, h).
3
428 pred hasMajorityEndorsement(ChaincodeInstance
529 ci) <--> count{orgEndorsesChaincode(_, ci)} >
600 count{orgParticipatesInChaincode(_, ci)} / 2.
71 error insufficientMajorityEndorsers
82 (ChaincodeInstance ci) <--> policy(ci, p),
9 MajorityPolicy(p),
103 !hasMajorityEndorsement(ci).

```

Listing 3 Layer 2: Mapping nodes to organizations.

In Listing 3 `hasMajorityEndorsement` predicate leverages Refinery’s `count{}` operator to verify that more than 50% of participating organizations provide endorsers, dynamically adapting to channel membership size. This enables early detection of policy violations before infrastructure planning (Layer 3). The `count{}` operator tallies distinct bindings: for a chaincode with 4 participating organizations, the predicate evaluates whether > 2 organizations provide endorsers. This dynamically adapts to channel membership changes without hardcoded thresholds.

Cross-layer relationships are central to incremental validation. The references `realizes/realizedBy` (Layer 0 to Layer 1), `endorses/endorsedBy` (Layer 2 to Layer 1), and `deployedOn` (Layer 2 to Layer 3) are first-class edges in the metamodel graph, highlighted in Figure 2 with green boxes marked with a bidirectional arrow. Because these are graph edges rather than external constraints, Refinery’s constraint propagation engine automatically re-evaluates all dependent predicates whenever a cross-layer edge is added or modified, enabling the incremental validation workflow described in Section 5.

```

321 //Enforce channel isolation
322 pred canEndorse(EndorsingNode n,
323 ChaincodeInstance ci) <-->
324 nodeParticipatesInChannel(n, ch), chaincodes
325 (ch, ci).
326
327 error CrossChannelEndorsement(n, ci) <-->
328 endorses(n, ci), !canEndorse(n, ci).

```

Listing 4 Layer 2 Preventing cross-channel endorsements

At this refinement stage, we validate: (1) each channel has multiple organizations (DLT requirement); (2) each channel has at least one orderer; (3) endorsement policies are satisfied by assigned nodes; and (4) no cross-channel violations occur. All validation completes before infrastructure planning begins at Layer 3.

Violation 3: Single-Organization Channel Detected at: Layer 2, after orderers are assigned and before infrastructure.

This violation occurs when an engineer creates a channel with only one participating organization, violating the fundamental blockchain requirement of multi-party collaboration.

Refinery detects `singleOrgChannel` (ChAudit) before planning infrastructure deployment, preventing the creation of channels that violate blockchain’s multi-party trust assumption.

Violation 4: Participation Mismatch Detected at: Layer 2, when nodes are assigned and before infrastructure.

This violation occurs when an orderer from an organization that does not participate in the `CollaborationIntent` is assigned to the channel.

Refinery detects `participantMismatch` (Regulator, ChNewOrders), ensuring channel participation matches business-level collaboration agreements. Detected before infrastructure deployment.

Violation 5: Cross-Channel Endorsement Detected at: Layer 2, when endorsers are assigned and before infrastructure.

This violation occurs when an endorsing node is assigned to a chaincode on a channel where its organization does not participate. Refinery detects `crossChannelEndorsement` (End_Manufacturer_Payment, CC_Payment), preventing nodes from endorsing transactions on channels they should not access (privacy violation).

Table 2 Violations Detected in Partial TPC-C Models

Violation Class	Layer	Example	Impact if Undetected
Unrealized Intent	L0–L1	Intent with no Channel	Forgotten requirement found at deployment
Single-Org Channel	L2	Channel with 1 Org	Invalid DLT topology; runtime failure
Policy Infeasibility	L1–L2	MajorityPolicy on 2-Org	Operational fragility; single-point failure
Participation Mismatch	L2	Orderer from non-member	Unauthorized access; security violation
Cross-Channel Leakage	L2	Endorser outside channel	Privacy violation; data leakage

In each case, the violation is caught before infrastructure planning (Layer 3), enabling immediate correction rather than costly late-stage redesign.

4.4. Layer3: Infrastructure

The final layer models the hardware constraints, including host machines (physical or virtual owned by any organization), network connectivity, and resource allocation. This is the final layer that completes the architecture specification.

```

1 class Host {
2 // Backlink to Layer 0
3 Organization[1] ^hosts opposite hosts}
4 class CommunicationNetwork {
5 Organization[] ownsNetworks}
6 class NetworkInterconnection {
7 CommunicationNetwork[2] connects}
8 // Validation: Nodes must be deployed
9 error undeclaredNode(Node n) <-->
10 Node(n), !deployedOn(n, _).

```

Listing 5 Layer 3 Physical reachability check.

As shown in Listing 5, `CommunicationNetwork` defines the network infrastructure connecting hosts, `NetworkInterconnection` class represents the connectivity between different networks, and the `deployedOn` relationship maps nodes to hosts.

This layer catches "Physical Unreachability." Because Layer 3 models `NetworkInterconnection`, Refinery can transitively verify if a Peer in Organization A can actually reach an Orderer in Organization B across different network segments.

With all layers specified, Refinery performs complete cross-layer validation where

- All Layer 0 intents realized
- All Layer 1 policies satisfied
- All Layer 2 nodes properly assigned
- All Layer 3 deployment complete

By decoupling the validation logic into these four discrete layers, the framework transforms the design process from a high-risk 'big bang' validation at the end of the project into a continuous, low-risk refinement pipeline. The complete 4-layer metamodel, TPC-C instance models, and the Refinery configuration used for this evaluation are available at: <https://github.com/ftsrg/policy-driven-hlf>.

5. Methodology

We present an incremental validation workflow that enables engineers to detect architectural violations as they refine models layer-by-layer, rather than deferring validation until complete specification exists.

5.1. Incremental Construction and Workflow

Building on the four-layer metamodel presented in Section 4, our approach enables engineers to build models layer-by-layer with immediate feedback at each step. Figure 3 shows the resulting workflow. The process alternates between activities (rounded rectangles) that introduce new design decisions and artifacts (document shapes) that represent the evolving partial model. Each layer transition triggers validation of the intra-layer and cross-layer predicates defined in (Sections 4.1, 4.2, 4.3, 4.4), ensuring that design decisions remain consistent with higher-layer requirements before proceeding to lower layers:

Layer 0 → Layer 1: Design network for consortium. Engineers specify organizations and collaboration intents, then map them to technical channels with endorsement policies. Refinery validates:

- `participantNotInConsortium`: All intent participants are consortium members
- `unrealizedIntent`: Every collaboration intent maps to a channel
- `conflictingPolicies`: Each chaincode has exactly one policy.

Layer 1 → Layer 2: Design logical topology for network. Engineers assign orderers and endorsers to channels. Refinery validates:

- `singleOrgChannel`: Each channel has multiple organizations
- `insufficientMajorityEndorsers`: Majority policies have sufficient endorsers
- `crossChannelEndorsement`: Nodes only endorse on their channels

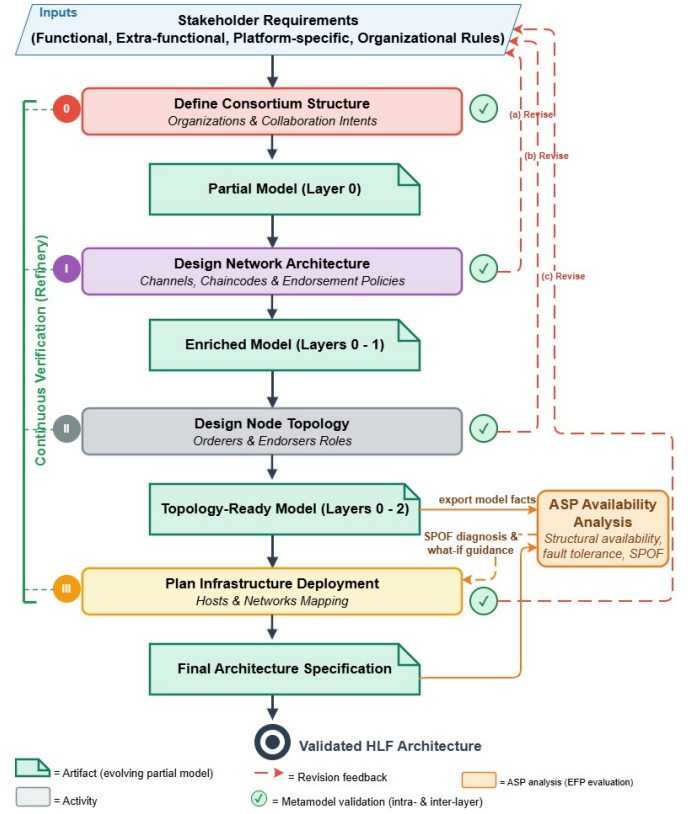


Figure 3 Four-layer refinement workflow

Layer 2 → Layer 3: Deploy topology to infrastructure. Engineers deploy nodes to hosts and specify network connectivity. Refinery validates:

- `undeployedNode`: All nodes are deployed to hosts

As shown in Figure 3, an ASP-based availability analysis complements Refinery's structural validation. After Refinery confirms structural validity at Layers 0-2, model facts are exported to the ASP solver, which computes minimum failure sets and identifies single points of failure. The ASP results feed back into the workflow as repair strategies (see Section 5.3), enabling engineers to iteratively improve both structural correctness and operational resilience. Refinery's metamodel uses bidirectional (**opposite**) relations to propagate constraints across layers:

- `ChaincodeInstance.endorsedBy` ↔ `EndorsingNode.endorses`
- `Channel.realizes` ↔ `CollaborationIntent.realizedBy`
- `Node.deployedOn` ↔ `Host.nodes`

Changes at any layer trigger reevaluation of dependent predicates at other layers, ensuring consistency throughout the refinement process.

5.2. Verification framework

Our ASP-based verification framework evaluates the dependency properties of HLF architectures. Verification can be performed not only on fully concrete models produced at the end of the refinement pipeline, but also on partial models at

intermediate refinement stages, enabling designers to assess the dependability implications of their decisions before committing to a complete architecture.

The primary dependability aspect we address is availability. To inform designers about the availability characteristics of their architecture, we compute a structural fault tolerance metric which we formalize as follows.

Definition 1 (Weakest-Link Fault Tolerance Score). *The weakest-link fault tolerance score of an HLF architecture is the minimum number of independent resource failures required to render at least one channel in the network unable to process transactions.*

A channel (c) is unavailable under a failure set (F) if (i) there exists a chaincode instance (ci) deployed on (c) such that the endorsement policy ($policy(ci)$) cannot be satisfied by the organizations whose endorsing peers remain operational under F , or (ii) if the ordering service for c has lost its consensus majority under F .

For a given endorsement policy, the fault tolerance characteristics differ. Under AllPolicy, the failure of any single participating organization's endorsing peer renders the chaincode instance unavailable. Under MajorityPolicy with n participating organizations, the chaincode instance tolerates up to $\lfloor (n-1)/2 \rfloor$ simultaneous organization-level failures. Under AnyPolicy, the chaincode instance remains available as long as at least one endorsing peer is operational. Since each channel may host multiple chaincode instances with different policies, the channel-level fault tolerance is determined by its most restrictive chaincode.

Listing 6 shows the core of our fault tolerance analysis. We introduce a combinatorial choice variable for each endorsing node, representing a possible failure. A node that has not failed is considered alive. The organization owning each node is derived from the deployment and host ownership relations exported from the Refinery model.

```

36 1 % Derived: all nodes
37 2 node(N) :- orderingnode(N).
38 3 node(N) :- endorsingnode(N).
39 4
40 5 % Choice: any node may fail
41 6 { failed(N) } :- node(N).
42 7
43 8 % Alive nodes
44 9 alive(N) :- node(N), not failed(N).
45 10
46 11 % ... Derived predicates
47 12
48 13 % AllPolicy violation
49 14 all_policy_violated(CC) :-
50 15     allpolicy(P), policy(CC, P),
51 16     org_participates_cc(Org, CC),
52 17     not org_can_endorse(Org, CC).
53 18
54 19 % MajorityPolicy violation
55 20 majority_policy_violated(CC) :-
56 21     majoritypolicy(P), policy(CC, P),
57 22     total_part_orgs(CC, T),

```

```

23     alive_endorse_orgs(CC, E), E * 2 <= T.
24
25 % ... Derived predicates
26
27 % Channel unavailability
28 channel_unavailable(Ch) :-
29     ordering_failed(Ch).
30 channel_unavailable(Ch) :-
31     chaincodes(Ch, CC),
32     cc_unavailable(CC).
33
34 % Constraint: at least one channel must become
35     unavailable
36 :- not some_unavailable.
37 some_unavailable :- channel_unavailable(_).
38
39 % Minimize failed nodes
40 #minimize { 1, N : failed(N) }.

```

Listing 6 Fault tolerance analysis in ASP.

Endorsement policy violations are checked at the organization level. An organization can still endorse a chaincode instance if it has at least one alive endorsing node assigned to that chaincode. For AllPolicy, a violation occurs when any participating organization loses its endorsement capability. For MajorityPolicy, the #count aggregate computes the number of participating and alive endorsing organizations; a violation occurs when the alive count does not exceed half of the total. A channel becomes unavailable if any of its chaincode instances has a policy violation.

The integrity constraint requires that at least one channel becomes unavailable, and the #minimize directive finds the smallest such failure set. This yields the *weakest-link* fault tolerance score: the minimum number of node failures required to disrupt any channel in the architecture.

In our example, the framework computes a single network-wide metric: the weakest-link fault tolerance, i.e., the minimum failure set that renders *any* channel unavailable. This is a deliberate design choice for the evaluation, but the ASP formulation generalizes naturally to richer analyses. By replacing the integrity constraint and optimization objective, the same fact base and derived predicates support per-channel and per-chaincode availability queries. The choice of metric and granularity depends on the designer's current concern: weakest-link identifies the most urgent single point of failure, per-channel analysis supports targeted hardening of critical channels. In each case, the ASP solver returns both the numeric score and the concrete failure set or coalition, providing the designer with actionable diagnostic information for the next refinement step.

5.3. Illustrative ASP analysis example

We demonstrate the ASP-based analysis on the smallest TPC-C model (five organizations, four channels). The model is structurally valid: Refinery reports no constraint violations across all four layers. However, structural validity does not imply operational resilience. The ASP analysis complements Refinery's structural validation by computing the weakest-link fault tolerance score.

The ASP solver identifies the minimum failure set:

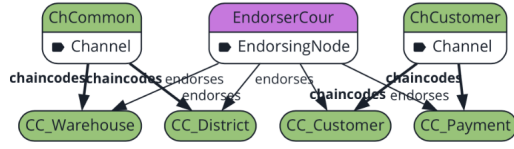


Figure 4 EndorserCour as single point of failure across two channels

```

1      failed(EndorserCour) ⇒
2      channel_unavailable(ChCustomer),
3      channel_unavailable(ChCommon)

```

A single node failure of the Courier organization’s endorsing peer renders *two* channels simultaneously unavailable. Figure 4 illustrates the shared vulnerability as the Courier organization participates in multiple channels under AllPolicy but provides only a single endorsing node to cover all of them.

Specifically, EndorserCour is the sole endorser for the Courier organization and endorses chaincodes on both ChCustomer (CC_Customer, CC_Payment) and ChCommon (CC_Warehouse, CC_District). Both channels require AllPolicy, meaning every participating organization must provide at least one active endorser. When EndorserCour fails, the Courier organization loses its endorsement capability entirely, violating AllPolicy on both channels simultaneously:

- **ChCustomer** (2 orgs: Retailer, Courier): The Courier can no longer endorse ⇒ AllPolicy violated.
- **ChCommon** (5 orgs: all members): The Courier can no longer endorse ⇒ AllPolicy violated, despite the other four organizations remaining fully operational.

This *correlated channel failure* is difficult to detect by manual inspection. A designer reasoning about ChCustomer in isolation would correctly identify its vulnerability (two organizations under AllPolicy), but would likely overlook that the same node is also critical for ChCommon (five-organization channel that *appears* resilient due to its larger membership). The vulnerability arises not from any single channel’s configuration, but from the combination of (1) an organization participating in multiple AllPolicy channels and (2) that organization providing no endorser redundancy.

The SPOF diagnosis triggers a *what-if* exploration where the engineer modifies the model and receives immediate feedback from Refinery (structural validity) and ASP (updated fault tolerance score). Table 3 summarizes three repair strategies.

Strategy A adds redundancy at the node level but both endorsers share HostCour, leaving a host-level SPOF. Strategy B eliminates this by deploying the second endorser on a new host (permitted by Host[1..*]). Strategy C relaxes the policy on ChCommon, tolerating up to two organization failures ($\lfloor (5-1)/2 \rfloor = 2$), but leaves ChCustomer vulnerable since AllPolicy on two organizations is inherently fragile which is a constraint rooted in the Layer 0 participation decision.

To measure the cost of interactive feedback during model refinement, we selected the first step of Strategy B (Table 3) as a representative refinement action. The first step is an

Table 3 Repair strategies for the EndorserCour SPOF

Strategy	Layer	Modification	Outcome
A	L2	Add EndorserCour2 on same host	Node resolved; SPOF host remains
B	L2+L3	Add EndorserCour2 on new HostCour2	Both node and host SPOF resolved
C	L1	ChCommon: AllPolicy → MajorityPolicy	ChCommon resilient; ChCustomer unchanged

infrastructure-level decision where the engineer assigns an unallocated host resource to the Courier organization. In our benchmark models, two additional hosts are present but not yet assigned to any organization, representing available infrastructure capacity within the consortium (e.g., reserve servers or cloud instances). The hosts relation between these extra hosts and existing organizations is UNKNOWN in Refinery’s partial model, reflecting the fact that the allocation decision has not yet been made. When the engineer commits to hosts(Courier, ExtraHost1) or hosts(Courier, ExtraHost2), Refinery propagates the consequences across all cross-layer predicates. We measure this single refinement step (one edge decision and its full propagation) as the unit of interactive feedback.

6. Evaluation

6.1. Benchmark Generation

In order to measure the performance of our approach, we extended the TPC-C supply chain example to a validation benchmark, which can be parameterized by the *size* (s) of the source model measured in *participating organizations*.

We have created valid supply chain models as follows:

1. A base consortium is created with *five* core organizations. These organizations establish *four* collaboration channels, see Section (3).
2. To model supply chain expansion, we add *Partner* organizations (*Partner1*, *Partner2*, ...) representing new suppliers or distributors joining the consortium. Each partner follows a realistic participation pattern:
 - Joins *ChCommon* (all members need reference data)
 - Joins *ChInventory* (tracking stock with existing members)
 - Does *not* join *ChNewOrders* or *ChCustomer* (business specific channels)
3. Step 2 is repeated ($s-5$) times, creating models with a total organizations where $s \in \{5, 6, 10, 15, 20\}$.
4. Each organization provides 1 *Host*, *Orders* for each channel they participate in (1-2 orders per org), *Endorsers* for all chaincodes on their channels (6-9 endorsers per org).

This scaling approach is realistic because it models organic growth of a supply chain consortium where new partners join existing collaboration structures without restructuring the entire network. The *ChNewOrders* and *ChCustomer* channels remain fixed at 3 and 2 organizations respectively, reflecting the

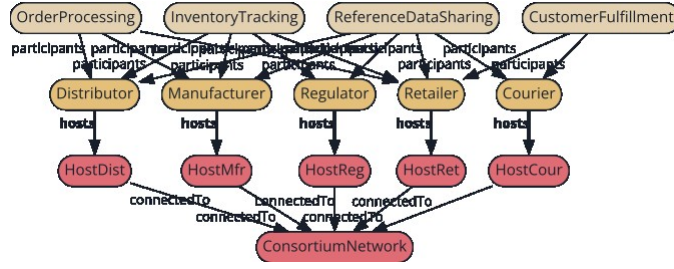


Figure 5 TPC-C Model Part 1

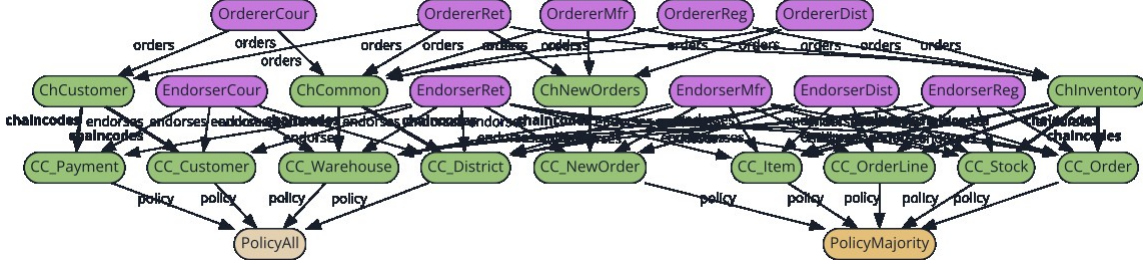


Figure 6 TPC-C Model Part 2

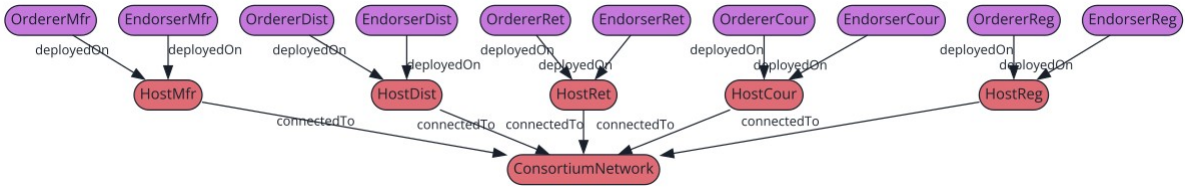


Figure 7 The Complete TPC-C Model, which connects 5 and 6

business reality that core transaction channels have stable membership while reference data sharing expands with the network.

The benchmark generation is automated by a Python script (available in the repository) that produces valid Refinery problem files for any target consortium size s . The script instantiates the base five-organization model and incrementally adds Partner organizations following the participation pattern described above, generating all necessary Layer 0–3 model elements (organizations, channels, nodes, hosts, endorsement assignments) for each new partner.

Ongoing work targets using Refinery’s own model generation capabilities for such parametric benchmark synthesis, which would additionally enable exploration of structurally diverse (Semeráth et al. 2020) configurations beyond the fixed expansion pattern. A Python script for parameterized benchmark generation is available in the repository (<https://github.com/ftsrg/policy-driven-hlf>).

6.2. Experimental Setup

We evaluated our approach on the TPC-C benchmark models with consortium sizes $s \in \{5, 6, 10, 15, 20\}$ organizations.

As the unit of interactive feedback, we use the first step of Strategy B (Table 3): the engineer commits randomly to `hosts(Courier, ExtraHost1)` or `hosts(Courier, ExtraHost2)`, assigning an unallocated host to the Courier

organization. Refinery propagates this single edge decision across all cross-layer predicates.

For each model size we report three quantities: (i) *Refinery re-validation time* per refinement step, (ii) *ASP fault tolerance analysis time*, and (iii) *pipeline time breakdown* separating one-time initialization (model loading and reasoning setup) from per-action refinement cost.

For each model size, we performed 100 measurements and report median values along with the full distribution. Each measurement session was preceded by 30 warm-up iterations whose results were discarded.

For Refinery model validation and ASP solving, the following computational configuration was used¹.

6.3. Results and Discussion

We revisit the three research questions posed in Section 1.2 and evaluate each against the experimental results from our TPC-C benchmark.

RQ1: Early Detection Capability Section 4 demonstrated five distinct violation classes on partial TPC-C models. Table 2 summarizes the detection layer for each class. Unrealized intents are caught at the Layer 0–1 transition, before any node

¹ OS: MacOS 26.2, CPU: 2.6 GHz 6-Core Intel i7, Memory: 32 GB 2667 MHz DDR4, Java VM: OpenJDK 64-Bit Server VM Corretto-21.0.2.13.1, Heap Space: 16GB

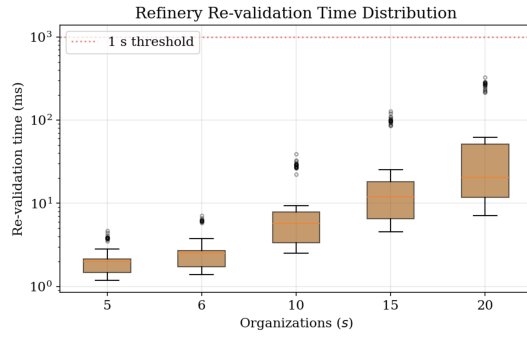


Figure 8 Distribution of Refinery re-validation time per refinement step

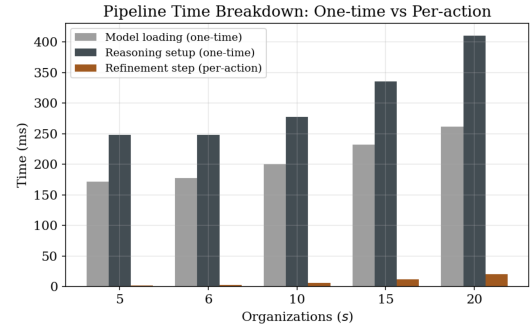


Figure 9 Pipeline time breakdown: one-time initialization (model loading and reasoning setup) versus per-action refinement cost.

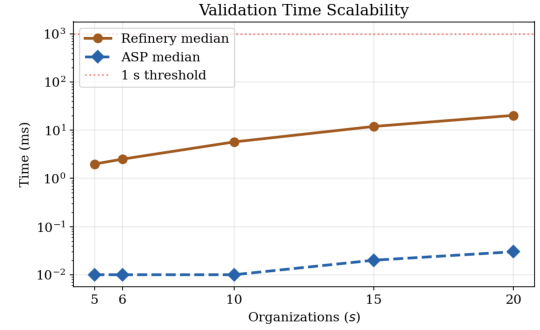


Figure 10 Validation time scalability across model sizes (logarithmic scale).

1 topology exists. Policy infeasibility (e.g., [MajorityPolicy](#) on a
2 two-organization channel) is flagged at Layers 1–2, when the
3 policy is assigned but endorsers have not yet been allocated.
4 Participation mismatches and cross-channel endorsements are
5 detected at Layer 2, once nodes are assigned to channels but
6 before any infrastructure commitment. In all five cases, the vio-
7 lation is identified and correctable before Layer 3 infrastructure
8 planning begins.

9 The illustrative ASP analysis further demonstrates that even
10 structurally valid models can harbor operational vulnerabilities.
11 The single point of failure is invisible to per-channel manual
12 reasoning but is surfaced automatically by the solver. This com-
13 plements the Refinery-based violation detection by covering depen-
14 dency concerns that go beyond structural well-formedness.

RQ1: The layered approach detects all five violation classes (unrealized intents, single-organization channels, policy infeasibility, participation mismatches, cross-channel endorsements) in partial models at Layers 0–2, before infrastructure planning begins at Layer 3. Each violation is caught at the earliest layer where sufficient information exists, enabling immediate correction rather than costly late-stage redesign.

16 **RQ2: Interactive Feedback** The intended workflow requires
17 that each refinement step produces feedback fast enough that
18 the engineer perceives it as instantaneous. Figure 8 shows the
19 distribution of Refinery re-validation times across model sizes.
20 The median remains in the 5–30 ms range for all configurations,
21 with worst-case tail latency below 200 ms even for the largest
22 model ($s=20$). The tail latency is attributable to constraint
23 propagation cascades where a single edge decision triggers re-
24 evaluation of predicates across multiple layers. No measurement
25 exceeded the 1-second interactive threshold.

26 Figure 9 places these per-action costs in workflow context
27 by separating one-time initialization from the repeated refine-
28 ment steps. Model loading and reasoning setup represent a
29 startup cost (200–400 ms depending on model size) incurred
30 once when the engineer opens the model. Each subsequent
31 refinement decision triggers re-validation at a cost consistently
32 two orders of magnitude lower than initialization. This sepa-
33 ration is central to the intended design workflow: the engineer
34 incurs the startup cost once, then explores multiple alternatives
35 through a sequence of incremental decisions, each receiving

near-instantaneous feedback.

RQ2: Re-validation time remains well below the 1-second interactive threshold for all model sizes tested (5–20 organizations). The median per-step cost is 5–30 ms. Worst-case latency stays below 200 ms. One-time initialization (200–400 ms) is amortized across all subsequent refinement steps, supporting continuous background validation without interrupting the design workflow.

RQ3: Scalability A key concern for practical adoption is whether validation remains interactive as the consortium grows. Figure 10 shows the scaling behavior of both validation components on a logarithmic scale from 5 to 20 organizations. ASP fault tolerance analysis time remains nearly constant across all model sizes (approximately 10^{-2} to 10^{-1} ms), because the search space is governed by the endorsement topology rather than the total number of model elements. Refinery re-validation time grows approximately linearly with model size but stays orders of magnitude below the 1-second threshold even at $s=20$.

The benchmark scales the model by adding partner organizations to [ChCommon](#) and [ChInventory](#), increasing the number of nodes, hosts, and cross-layer predicates evaluated per refinement step. Despite this growth, both components remain well within the interactive range, indicating that the approach can support consortia of at least 20 organizations. It covers the range typical of enterprise blockchain deployments such as trade finance networks or supply chain consortia.

RQ3: Both validation components scale favorably from 5 to 20 organizations. ASP analysis time remains nearly constant; Refinery re-validation grows approximately linearly but stays orders of magnitude below the interactive threshold. The approach supports real-time feedback for consortium sizes typical of enterprise blockchain deployments.

6.4. Threats to Validity

We identify the following threats to the validity of our approach and experimental results:

Internal Validity. We ran each experiment 100 times and report median values along with the full distribution, and we tested different random bases for the solver.

External Validity. While our layers are designed for HLF, the specific technical entities (e.g., peers, orderers) are platform-specific. The general refinement methodology may require adaptation for other DLT platforms like Ethereum (Baresi et al. 2022) or Raft-based systems (Ongaro & Ousterhout 2014).

Our running example uses a TPC-C supply chain consortium. While it captures common architectural challenges including multi-channel participation, differentiated endorsement policies it may not fully represent every edge case in massive-scale industrial networks. We focused on three endorsement policy types (*AllPolicy*, *MajorityPolicy*, *AnyPolicy*). More complex endorsement policies supported by Hyperledger Fabric, such as custom policy expressions combining AND/OR logic across organizational subsets, are not yet modeled in our framework.

Construct Validity. We validated our metamodel against the official HLF documentation and cross-checked all predicates with blockchain experts. All error predicates were tested with known invalid configurations to ensure they fire correctly.

While our framework detects five classes of architectural violations (Section 4), other violation types relevant to production deployments (e.g., performance bottlenecks, security vulnerabilities) are not currently addressed.

7. Related Work

Hyperledger Fabric modeling tools. As reflected by the survey of Curty et al. on the use of MDE in model-based applications (Curty et al. 2023), currently, model-driven approaches are dominantly employed for smart contract design. Specifically for BPMN and Ethereum-based multi-chain deployment, early, architecturally relevant results are appearing (Bodorik et al. 2023), but these do not approach “architecture” as a true first-class concept and, to our knowledge, have not been translated to Hyperledger Fabric yet. Nevertheless, the architecture of the system, including the number of channels and nodes, remains the determining factor in the performance and reliability of the HLF deployment (Toumia et al. 2021).

Logically, the deployment automation needs of (consortial) blockchains should lead to at least DSLs relevant to architecture *description*. Górski & Bednarski (2020) propose a UML profile for describing blockchain deployments and define validations for comparing the UML model with the deployed node configurations. However, they do not target validations for the blockchain deployment model itself or the architecture at large.

Barišić et al. (2021) propose a layered modeling language for describing multi-chain smart contract deployments. They define a workflow layer, a mapping layer, and an implementation layer that targets various blockchain platforms. However, as they focus on public blockchains like Tezos and Ethereum, they do not model consortial aspects or node and hardware configurations.

For Ethereum, Baresi et al. (2022) introduce KATENA, a framework that simplifies the deployment and management of blockchain applications. For Fabric, the Hyperledger Labs project Fablo uses declarative network descriptions in JSON for deployment. For deployment on Kubernetes, Helm charts are also available.

Uncertain models. Partial models are similar to uncertain models, which offer a rich specification language (Famelis et al. 2012; Salay & Chechik 2015) amenable to analysis. They provide a more intuitive, user-friendly language compared to 3-valued interpretations, which allows the designers to annotate existing models with uncertainty (e.g. weather an object may or may not exist). This allows the developer to reuse existing models as initial designs plans, but they cannot be used to specify requirements. Moreover, uncertain models often need to be formatted as valid models, which limits the set of partial structures that can be described. Additionally, uncertain models does not handle WF constraints natively.

Potential concrete models compliant with an uncertain model can be synthesized by the Alloy Analyzer (Salay et al. 2012), or refined by graph transformation rules (Salay et al. 2015). Approaches like (Famelis et al. 2013) analyze possible matches and executions of model transformation rules on partial models by using a SAT solver (MathSAT4) or by automated graph approximations using graph query engines (Marussy et al. 2020). **Logic-based partial modeling.** Different approaches use abstract interpretation (Rensink & Distefano 2006), or predicate abstraction (Reps et al. 2004; Ferrara et al. 2012; Gopan et al. 2004) for partial modeling. In those approaches, concretization is used to materialize (typically small) counter-examples for designated safety properties in a graph transformation system. However, their focus is to support model checking of abstract graph transformation systems, which can evaluate complex trajectories, but do not scale in the size of the models.

In contrast, our approach leverages Refinery (Marussy et al. 2024) and ASP (Calimeri et al. 2019) as logical partial modeling and analysis tools to enable the validation of large-scale Hyperledger Fabric consortia, with a focus on structural and reliability aspects.

Generative architectures with Design Space Explorers. Graph-based Design-Space Exploration (DSE) use graph transformations (Agrawal et al. 2002) or refactorings to generate candidate designs as graph models. They are widely used to automatically evaluate extra-functional characteristics of architecture models, and synthesize architecture proposals. They either rely on *model-based* search, where a graph model is mutated, or *rule-based* search, where solutions constructed as a sequence of graph transformations (John et al. 2019). As a defining characteristic, architecture models are formalized as instances of a modeling languages. Therefore, metamod-

els, such as the one we present for a multi-layer description of Hyperledger Fabric consortia, are a key component in such approaches.

MOMoT (Fleck et al. 2016) and MDEOptimiser (Burdusel et al. 2018) rely on the Henshin model transformation language (Strüber et al. 2017) for model-based exploration. VIATRA-DSE (Abdeen et al. 2014; Hegedüs et al. 2015) is a rule-based DSE tool that relies on the VIATRA (Ujhelyi et al. 2015) language. SHEPhERd (Cortellessa et al. 2015) and EASIER (Arcelli et al. 2018) aim to derive sequences software architecture refactorings according to extra-functional criteria. Földiák et al. (2022) rely on external numerical solvers to calculate and optimise dependability metrics in both rule-based DSE and logic-based partial modeling.

While our present work does not focus on the automated synthesis of blockchain architectures, but the presented validation and reliability evaluation techniques could form the basis of the hard constraints and objective functions in a tool for generative architectures.

8. Conclusions

We presented a four-layer modeling framework for the incremental design and validation of Hyperledger Fabric architectures. By decomposing architectural decisions into consortium structure (Layer 0), network architecture (Layer 1), node topology (Layer 2), and infrastructure deployment (Layer 3), our approach enables validation of partial designs at each refinement step rather than deferring all checks to deployment time.

Using Refinery’s partial-model reasoning, we demonstrated detection of five classes of architectural violations unrealized intents, single-organization channels, endorsement policy infeasibility, participation mismatches, and cross-channel endorsements—all caught at Layers 0–2 before infrastructure planning begins. An ASP-based fault tolerance analysis complements structural validation by identifying shared vulnerabilities, such as the EndorserCour single point of failure that simultaneously affects two channels under AllPolicy.

Our evaluation on TPC-C supply chain benchmarks with 5 to 20 organizations shows that re-validation time remains in the 5–30 ms range per refinement step, well below the 1-second interactive threshold. Both Refinery and ASP analysis scale favorably, supporting real-time feedback for consortium sizes typical of enterprise blockchain deployments.

References

Abdeen, H., Varró, D., Sahraoui, H., Nagy, A. S., Debreceni, C., Hegedüs, Á., & Horváth, Á. (2014). Multi-objective optimization in rule-based design space exploration. In *Ase* (p. 289-300). ACM.

Agrawal, A., Levendovszky, T., Sprinkle, J., Shi, F., & Karsai, G. (2002). Generative programming via graph transformations in the model-driven architecture. In *Workshop on generative techniques in the context of model driven architecture, oopsla*.

Al-Gburi, N. M. S., Földvári, A., Marussy, K., Semeráth, O., & Kocsis, I. (2024). Requirement-driven generation

of distributed ledger architectures. In *Proceedings of the acm/ieee 27th international conference on model driven engineering languages and systems* (p. 268–279). New York, NY, USA: Association for Computing Machinery. Retrieved from <https://doi.org/10.1145/3640310.3674097> doi: 10.1145/3640310.3674097

Androulaki, E., De Caro, A., Neugschwandtner, M., & Sorniotti, A. (2019). Endorsement in hyperledger fabric. In *2019 ieee international conference on blockchain (blockchain)* (p. 510–519). doi: 10.1109/Blockchain.2019.00077

Arcelli, D., Cortellessa, V., D’Emidio, M., & Pompeo, D. D. (2018). EASIER: An evolutionary approach for multi-objective software architecture refactoring. In *Isca* (p. 105–114). IEEE.

Baresi, L., Quattrocchi, G., Tamburri, D. A., & Terracciano, L. (2022). A declarative modelling framework for the deployment and management of blockchain applications. In *Models*. ACM.

Barišić, A., Zhu, E., & Mallet, F. (2021). Model-driven approach for the design of multi-chain smart contracts. In *Brains*.

Bodorik, P., Liu, C. G., & Jutla, D. (2023). TABS: Transforming automatically BPMN models into blockchain smart contracts. *Blockchain: Research and Applications*, 4(1), 100115.

Burdusel, A., Zschaler, S., & Strüber, D. (2018). MDEoptimiser: A search based model engineering tool. In *Models* (p. 12-16). ACM.

Calimeri, F., Faber, W., Gebser, M., Ianni, G., Kaminski, R., Krennwallner, T., ... Schaub, T. (2019). ASP-core-2 input language format. *Theor. Practice Logic*, 20(2), 294–309.

Cortellessa, V., Mirandola, R., & Potena, P. (2015). Managing the evolution of a software architecture at minimal cost under performance and reliability constraints. *Sci. Comput. Program.*, 98, 439–463.

Curty, S., Härer, F., & Fill, H.-G. (2023). Design of blockchain-based applications using model-driven engineering and low-code/no-code platforms: a structured literature review. *Software and Systems Modeling*, 1–39.

DiMatteo, L. A., Cannarsa, M., & Poncibò, C. (2019, 10). The cambridge handbook of smart contracts, blockchain technology and digital platforms. *The Cambridge Handbook of Smart Contracts, Blockchain Technology and Digital Platforms*, 1-366. Retrieved from <https://www.cambridge.org/core/books/cambridge-handbook-of-smart-contracts-blockchain-technology-and-digital-platforms/BCDDFAAD7B661E6C268941ACA76B3A58> doi: 10.1017/9781108592239

Famelis, M., Salay, R., & Chechik, M. (2012). Partial models: Towards modeling and reasoning with uncertainty. In *ICSE* (pp. 573–583). IEEE.

Famelis, M., Salay, R., Di Sandro, A., & Chechik, M. (2013). Transformation of models containing uncertainty. In *International conference on model driven engineering languages and systems* (pp. 673–689).

Ferrara, P., Fuchs, R., & Juhasz, U. (2012). TVAL+: TVLA and value analyses together. In *Sefm 2012* (Vol. 7504, p. 63-77). Springer.

- 1 Fleck, M., Troya, J., & Wimmer, M. (2016). Search-based
2 model transformations with MOMoT. In *Icmt* (Vol. 9765,
3 p. 79-87). Springer.
- 4 Földiák, M., Marussy, K., Varró, D., & Majzik, I. (2022).
5 System architecture synthesis for performability by logic
6 solvers. In *Models* (p. 43-54). ACM.
- 7 Gelfond, M., & Lifschitz, V. (1988). The stable model semantics
8 for logic programming. In *Iclp/slp* (Vol. 88, pp. 1070-1080).
- 9 Gopan, D., DiMaio, F., Dor, N., Reps, T., & Sagiv, M. (2004).
10 Numeric domains with summarized dimensions. In *Tacas*
11 *2004* (Vol. 2988, p. 512-529). Springer.
- 12 Górski, T., & Bednarski, J. (2020). Applying model-driven
13 engineering to distributed ledger deployment. *IEEE Access*,
14 8.
- 15 Hegedüs, Á., Horváth, Á., & Varró, D. (2015). A model-driven
16 framework for guided design space exploration. *Autom. Softw.*
17 *Eng.*, 22(3), 399-436.
- 18 John, S., Burdusel, A., Bill, R., Strüber, D., Taentzer, G.,
19 Zschaler, S., & Wimmer, M. (2019). Searching for opti-
20 mal models: Comparing two encoding approaches. *J. Object*
21 *Technol.*, 18(3), 6:1-22.
- 22 Lifschitz, V. (2019). *Answer set programming*. Springer Berlin.
- 23 Marussy, K., Ficsor, A., Semeráth, O., & Varró, D. (2024).
24 Refinery: Graph solver as a service: Refinement-based gen-
25 eration and analysis of consistent models. In *Proceedings*
26 *of the 2024 ieee/acm 46th international conference on soft-*
27 *ware engineering: Companion proceedings* (p. 64-68). New
28 York, NY, USA: Association for Computing Machinery. Re-
29 trieved from <https://doi.org/10.1145/3639478.3640045> doi:
30 10.1145/3639478.3640045
- 31 Marussy, K., Semeráth, O., Babikian, A. A., & Varró, D. (2020).
32 A specification language for consistent model generation
33 based on partial models. *J. Obj. Technol.*, 19.
- 34 Ongaro, D., & Ousterhout, J. (2014). In search of an under-
35 standable consensus algorithm. In *Proceedings of the 2014*
36 *usenix conference on usenix annual technical conference*
37 (p. 305-320). USA: USENIX Association.
- 38 Rensink, A., & Distefano, D. (2006). Abstract graph transfor-
39 mation. *Electronic Notes in Theoretical Computer Science*,
40 157(1), 39-59.
- 41 Reps, T. W., Sagiv, M., & Wilhelm, R. (2004). Static program
42 analysis via 3-valued logic. In *Cav* (pp. 15-30).
- 43 Salay, R., & Chechik, M. (2015). A generalized formal frame-
44 work for partial modeling. In A. Egyed & I. Schaefer (Eds.),
45 *Fase* (Vol. 9033, p. 133-148). Springer.
- 46 Salay, R., Chechik, M., Famelis, M., & Gorzny, J. (2015). A
47 methodology for verifying refinements of partial models. *J.*
48 *Obj. Technol.*, 14(3), 3:1-31.
- 49 Salay, R., Famelis, M., & Chechik, M. (2012). Language
50 independent refinement using partial modeling. In *FASE* (pp.
51 224-239). Springer.
- 52 Semerath, O., Babikian, A. A., Pilarski, S., & Varro, D. (2019,
53 5). Viatra solver: A framework for the automated gener-
54 ation of consistent domain-specific models. *Proceedings*
55 *- 2019 IEEE/ACM 41st International Conference on Soft-*
56 *ware Engineering: Companion, ICSE-Companion 2019*, 43-
57 46. Retrieved from [doi: 10.1109/ICSE-Companion.2019](https://doi.org/10.1109/ICSE-Companion.2019.00034?download=true)
58 [.00034?download=true](https://doi.org/10.1109/ICSE-Companion.2019.00034?download=true) doi: 10.1109/ICSE-Companion.2019
59 [.00034](https://doi.org/10.1109/ICSE-Companion.2019.00034?download=true)
- 60 Semeráth, O., Farkas, R., Bergmann, G., & Varró, D. (2020).
61 Diversity of graph models and graph generators in mutation
62 testing. *Int. J. Softw. Tools Technol. Transf.*, 22(1), 57-78.
- 63 Strüber, D., Born, K., Gill, K. D., Groner, R., Kehrer, T., Ohrn-
64 dorf, M., & Tichy, M. (2017). Henshin: A usability-focused
65 framework for EMF model transformation development. In
66 *Icgt@staf* (Vol. 10373, p. 196-208). Springer.
- 67 Sukhwani, H., Wang, N., Trivedi, K. S., & Rindos, A. (2018).
68 Performance modeling of hyperledger fabric (permissioned
69 blockchain network). In *2018 ieee 17th international sympo-*
70 *sium on network computing and applications (nca)* (p. 1-8).
71 doi: 10.1109/NCA.2018.8548070
- 72 Tournia, S. B., Berger, C., & Reiser, H. P. (2021). Evaluating
73 blockchain application requirements and their satisfaction in
74 Hyperledger Fabric. In *DAIS*.
- 75 Transaction Processing Performance Council. (2010). *TPC-C*
76 *Benchmark Specification, Version 5.11* (Tech. Rep.). TPC.
77 Retrieved from [http://www.tpc.org/tpc_documents_current](http://www.tpc.org/tpc_documents_current_versions/pdf/tpc-c_v5.11.0.pdf)
78 [_versions/pdf/tpc-c_v5.11.0.pdf](http://www.tpc.org/tpc_documents_current_versions/pdf/tpc-c_v5.11.0.pdf)
- 79 Ujhelyi, Z., Bergmann, G., Hegedüs, Á., Horváth, Á., Izsó,
80 B., Ráth, I., ... Varró, D. (2015). EMF-INCQUERY: An
81 integrated development environment for live model queries.
82 *Sci. Comput. Program.*, 98(1), 80-99.